

Frontend : Theory Assignment 2

Name: VIKASH RAI

Section: NA(44)

Roll No: 22515000162

Q1. Explain the difference between margin and padding in CSS with an example.

Both margin and padding create space around an element, but they are placed in different areas. Padding is the space between the content and the border, while margin is the space outside the border.

- **Padding:** Increases the internal space of an element.
- **Margin:** Creates space between the element and surrounding elements.
- Padding is part of the element's box area, while margin separates neighboring elements.

Example:

```
div {  
  padding: 20px;  
  margin: 15px;  
}
```

Q2. Why is responsive web design important? How can CSS help achieve it?

Responsive web design allows a website to adjust its layout according to the available screen size. This is important because users may open the same website on mobiles, tablets and desktop computers.

- **Flexible layouts:** Elements can resize according to the screen.
- **Media queries:** Different styles can be applied at different viewport widths.
- **Flexible images:** Images can be prevented from overflowing their containers.
- **Better usability:** Users can read and interact with the page more easily on different devices.

Example:

```
@media (max-width: 700px) {  
  .menu {  
    display: block;  
  }  
}
```

Q3. What is Bootstrap's grid system? Explain the role of rows and columns.

Bootstrap provides a responsive grid system based on 12 units. A row is used to arrange columns horizontally, and the column classes decide how much of the row each section occupies.

- **Container:** Provides a main area for the grid.
- **Row:** Groups columns into a horizontal layout.
- **Column:** Occupies a selected number of the 12 grid units.
- Different column classes can be used for different screen sizes.

Example:

```
<div class="container">  
  <div class="row">  
    <div class="col-md-8">Main</div>  
    <div class="col-md-4">Side</div>  
  </div>  
</div>
```

Q4. How can Bootstrap buttons and cards be created? Give a simple example.

Bootstrap provides predefined classes that can be applied to HTML elements to quickly create styled components. Buttons and cards are common examples.

- **Buttons:** Classes such as `btn` and `btn-primary` provide a ready-made button style.
- **Cards:** Classes such as `card`, `card-body` and `card-title` help create content containers.

Example:

```
<button class="btn btn-primary">Submit</button>

<div class="card">
  <div class="card-body">
    <h5 class="card-title">Project</h5>
    <p class="card-text">Frontend work</p>
  </div>
</div>
```

Q5. Differentiate between class selectors and ID selectors in CSS.

A class selector is generally used when the same style needs to be applied to multiple elements. An ID selector targets an element with a particular id and is normally intended for a unique element.

- **Class:** Written with a dot, such as `.box`.
- **ID:** Written with a hash symbol, such as `#header`.
- Several elements can share the same class, whereas an id is normally assigned to one unique element on a page.

Example:

```
.box {
  padding: 10px;
}

#header {
  font-size: 24px;
}
```

Q6. How do background-image, background-size and background-position work together?

These properties control different parts of a background image. `background-image` selects the image, `background-size` controls its displayed size, and `background-position` controls where it is placed.

- **background-image:** Adds the image.
- **background-size:** Values such as `cover` or `contain` control scaling.
- **background-position:** Values such as `center` or `top right` control placement.

Example:

```
body {
  background-image: url("photo.jpg");
  background-size: cover;
  background-position: center;
}
```

Q7. What is z-index in CSS? When is it useful?

The `z-index` property controls the stacking order of positioned elements when they overlap. An element with a higher stacking value can appear in front of another element with a lower value, when the relevant stacking contexts allow it.

- It is useful for menus, pop-ups, badges, overlays and other overlapping interface elements.
- The elements generally need a positioning context for `z-index` to have the intended effect.

Example:

```
.menu {
  position: relative;
  z-index: 5;
}

.overlay {
  position: absolute;
  z-index: 10;
}
```

Q8. What is the difference between relative, absolute, fixed and sticky positioning?

The four positioning modes change how an element is placed and how it behaves during scrolling.

- **Relative:** Keeps its normal place and can be moved from that position.
- **Absolute:** Is removed from normal flow and positioned relative to a suitable positioned ancestor.
- **Fixed:** Is positioned relative to the viewport and normally stays visible during scrolling.
- **Sticky:** Behaves like a normally positioned element until a scrolling threshold is reached, then sticks within its containing area.

Q9. How is ECMAScript related to JavaScript? Mention some modern ECMAScript features.

ECMAScript is the standardized language specification, while JavaScript is the best-known implementation of that specification. JavaScript engines use ECMAScript rules to provide the language features used by developers.

- **let and const:** Block-scoped variable declarations.
- **Arrow functions:** Shorter function syntax.
- **Classes:** Syntax for defining class-based structures.
- **Promises:** Used for handling asynchronous operations.
- **Modules:** Allow code to be separated into reusable files.

Q10. Explain hoisting in JavaScript and compare the behavior of var, let and const.

Hoisting describes the way JavaScript processes declarations before executing the code in a scope. The behavior is different for var, let and const.

- **var:** Its declaration is hoisted and the variable is initialized with undefined before its assignment.
- **let:** Its declaration is hoisted but it cannot be accessed before the declaration is evaluated.
- **const:** Similar to let regarding the temporal dead zone, and it must be initialized when declared.
- **Temporal Dead Zone:** The period before a let or const declaration is initialized in which accessing the variable causes a ReferenceError.

Example:

```
console.log(a); // undefined
var a = 5;

console.log(b); // ReferenceError
let b = 10;
```